TRYHACKME · TUTEDUDE CYBERSECURITY TRACK · TASK 09

Reset Password

OTP Brute Force via Missing Rate Limiting

Platform: TryHackMe **Room:** tutedude-cybersec **Category:** Broken Authentication
Prepared by: Farhan **Date:** August 16, 2026

FLAG CAPTURED

FLAG{BRUTE_FORCE_BYPASS_SUCCESS}

Objective

This task targets the SecureCorp Portal's password-reset flow, with the goal of finding a flaw in the “forgot password” feature that lets an attacker change the password of an arbitrary account — specifically the admin account — without ever knowing its current credentials. The flow gates the reset behind a 4-digit OTP sent to the target account; this task focuses on whether that OTP is adequately protected against automated guessing, and demonstrates that it is not.

Environment & Tools

Target	SecureCorp Portal — http://192.168.155.129/forgot_password.php (TryHackMe lab instance, v1.0.1-beta)
Platform	TryHackMe — tutedude-cybersec room
Tools Used	ffuf v2.2.1, Browser DevTools (Application → Storage → Cookies), Chromium-based browser
Wordlist	SecLists — Fuzzing/4-digits-0000-9999.txt

Methodology

Step 1 — Initiating the Password Reset

Testing began at /forgot_password.php, which asks only for a username and, per the on-page copy, sends a 4-digit verification code to the associated account. “admin” was submitted as the target username to start a recovery flow for the highest-privilege account on the portal.

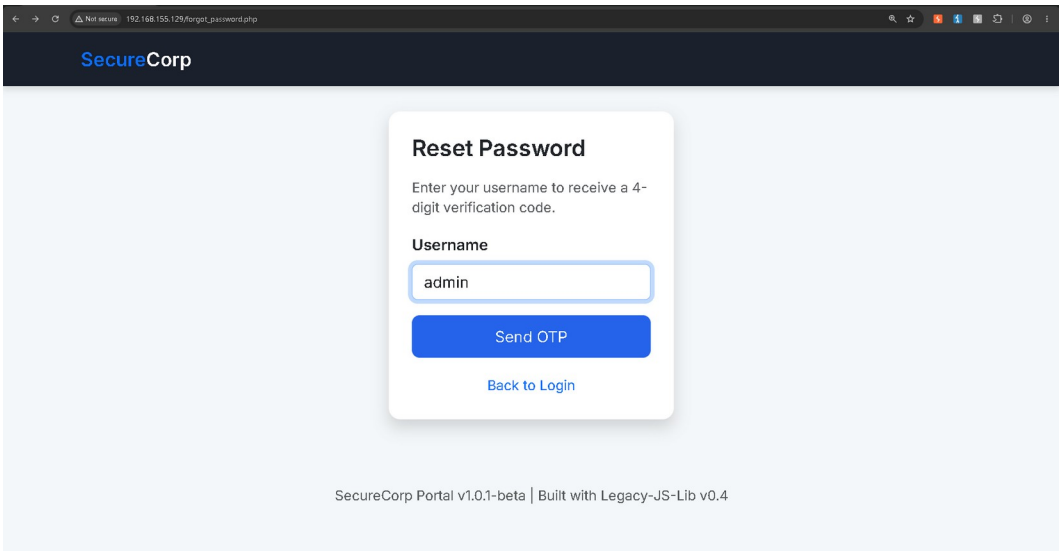

Figure 1 — forgot_password.php: initiating an OTP-based reset for the 'admin' account.

Step 2 — Inspecting the Session and the OTP Prompt

Submitting the form redirected to /verify_otp.php, which prompts for the 4-digit code and — notably — discloses its own weakness directly on the page:

```
Hint: In this lab, codes do not expire and there is no attempt limit.
```

Browser DevTools (Application → Storage → Cookies) confirmed the only session artifact in play was a standard PHPSESSID cookie, which would need to be replayed on every guess against the endpoint.

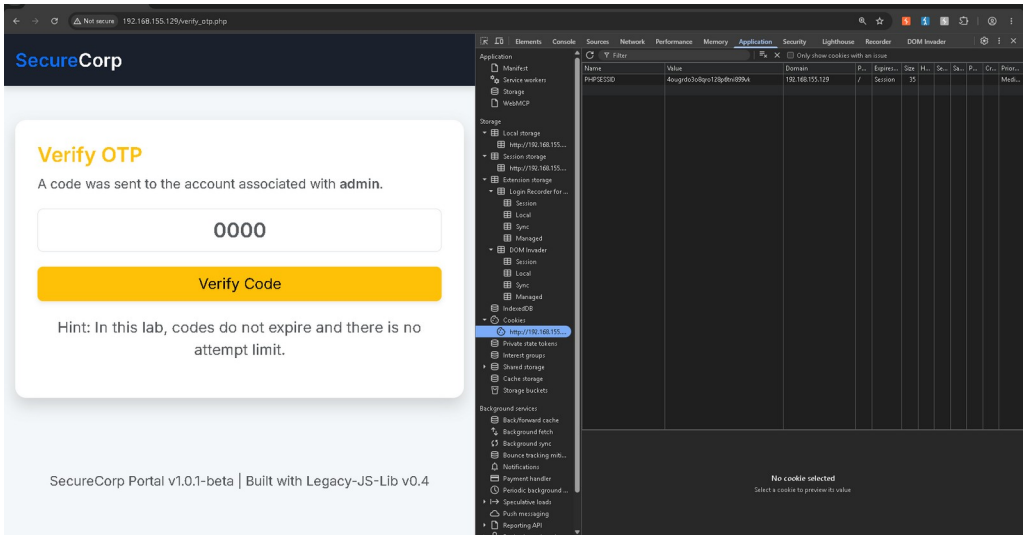

Figure 2 — verify_otp.php and DevTools: the page states outright that OTP codes never expire and carry no attempt limit.

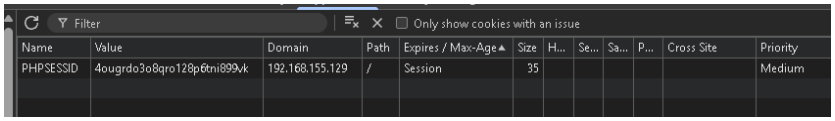

Figure 3 — Active PHPSESSID session cookie captured for reuse in the brute-force request.

Step 3 — Brute-Forcing the OTP with ffuf

With the session cookie in hand and no rate limiting to work around, the full 4-digit keyspace was exhaustively fuzzed with ffuf — posting every value from 0000 to 9999 to /verify_otp.php and filtering out the baseline "incorrect code" response size of 2,842 bytes:

```
ffuf -u http://192.168.155.129/verify_otp.php `
     -X POST `
     -b "PHPSESSID=4ougrdo3o8qro128p6tni899vk" `
     -H "Content-Type: application/x-www-form-urlencoded" `
     -d "otp=FUZZ" `
     -w C:\wordlists\SecLists\Fuzzing\4-digits-0000-9999.txt `
     -fs 2842
```

Of the 10,000 requests — completed in roughly five seconds at ~1,850 req/sec with zero errors — exactly one fell outside the filtered baseline:

```
1337   [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 0ms]
```

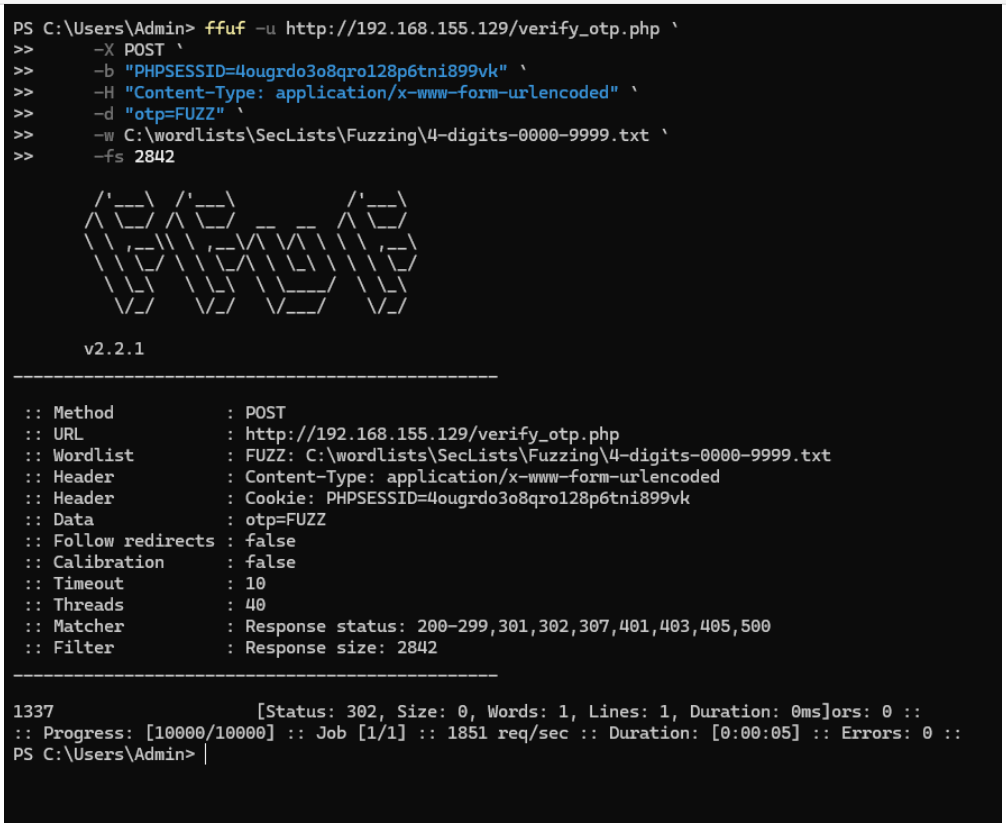

Figure 4 — ffuf isolating OTP 1337 as the sole response outside the calibrated baseline size.

Step 4 — Submitting the Valid OTP and Resetting the Password

Entering 1337 on /verify_otp.php passed verification with no additional check and landed on the account-recovery confirmation screen. The page confirmed the password reset had gone through for admin and revealed the objective flag; it also included a researcher note clarifying that, as a lab-mechanics detail, password changes are not persisted to the environment's static mock database, so the originally discovered credentials remain valid for any further access.

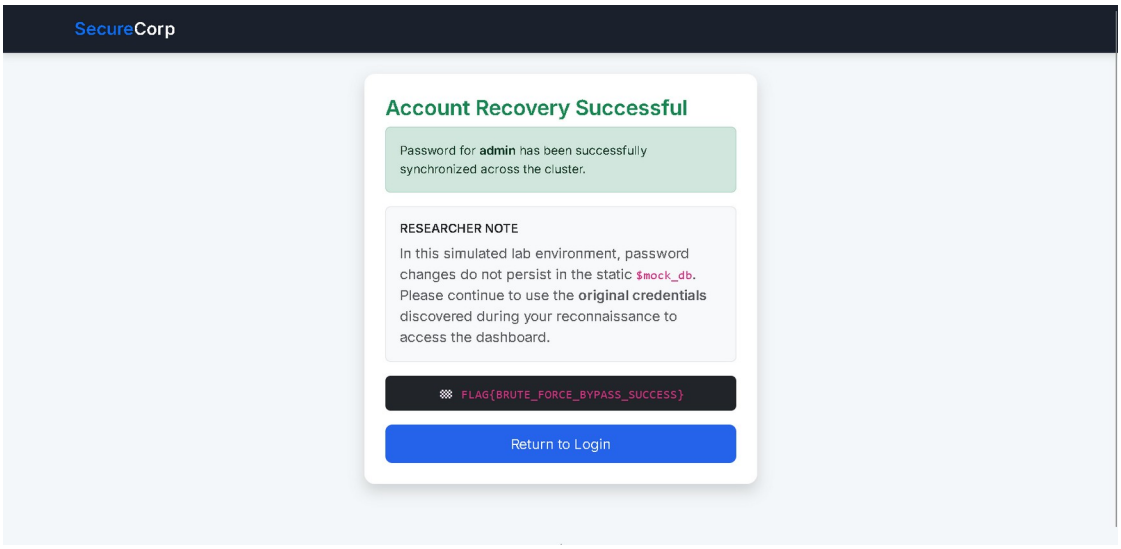

Figure 5 — Account Recovery Successful: password reset confirmed for 'admin', flag revealed.

Step 5 — Flag Capture & Validation

FLAG{BRUTE_FORCE_BYPASS_SUCCESS} was submitted to the lab platform's answer field and validated as correct, closing out the objective.

Figure 6 — Flag accepted on the TryHackMe / Tutedude lab platform.

Vulnerability Analysis

Vulnerability	Improper Restriction of Excessive Authentication Attempts — password-reset OTP brute-forced end to end
OWASP Category	A07:2021 – Identification and Authentication Failures
Weakness (CWE)	CWE-307: Improper Restriction of Excessive Authentication Attempts (related: CWE-640, Weak Password Recovery Mechanism)
Severity	Critical — unauthenticated takeover of any account, including admin, via the password-reset flow alone

The root cause is twofold: the OTP issued by /verify_otp.php never expires, and the endpoint places no limit on verification attempts for a given session. Because the code space is only four digits — 10,000 possible values — both weaknesses together turn what should be a one-time, time-boxed secret into a value that can be exhaustively searched with an ordinary fuzzing tool. ffuf needed only the active PHPSESSID and a stock 4-digit wordlist to cover the entire keyspace in about five seconds, distinguishing the one correct code by its distinct HTTP response — a 302 redirect with a zero-byte body — against the calibrated size of every incorrect attempt.

Impact

- Complete takeover of any account, including admin, through the password-reset flow alone — no prior credentials, phishing, or social engineering required.
- The entire 10,000-value OTP keyspace was exhausted in about five seconds against a single session, at roughly 1,850 requests/second, with zero errors.
- Because the reset flow needs only a username, and “admin” is a near-universal default account name, the flaw effectively removes authentication as a control for the highest-privilege account.
- No CAPTCHA, progressive delay, lockout, or step-up check exists between attempts, so exploitation requires nothing beyond a publicly available fuzzing tool.
- Response size alone (2,842 bytes for a wrong code vs. 0 bytes for the correct one) was enough to fingerprint success, making the attack fully automatable and unambiguous.

Remediation Recommendations

1. Rate-limit and lock the OTP verification endpoint after a small number of failed attempts (e.g., 5) per account and per session/IP, with exponential backoff or a temporary cool-down.
2. Expire every OTP after a short, fixed window (5–10 minutes) and invalidate it immediately after first use or whenever a new code is requested.
3. Increase OTP entropy — a 4-digit numeric code is inherently guessable; move to a longer, cryptographically random alphanumeric token.
4. Return uniform status codes, response sizes, and timing for both valid-format-but-wrong and invalid submissions, so response characteristics can't be used to fingerprint the correct value.
5. Bind account-recovery actions to additional context (e.g., a confirmed email/SMS channel plus re-authentication) rather than a short numeric code alone.
6. Log and alert on repeated OTP failures for the same session or target account, and treat unusual verification velocity as an active brute-force indicator rather than passing it through silently.

Conclusion

This task demonstrates a textbook authentication-failure vulnerability (OWASP A07:2021): the password-reset flow gated a sensitive action — resetting any account’s password — behind a 4-digit OTP with no expiry and no attempt limiting. Because the correct code could be isolated purely from response size using an off-the-shelf fuzzer, the full 10,000-value keyspace was exhausted in seconds and the admin account’s password was reset without ever knowing its original credentials. The exercise reinforces that account-recovery flows deserve the same authentication rigor as login itself — rate limiting, short-lived codes, and sufficient entropy are not optional extras.

FLAG{BRUTE_FORCE_BYPASS_SUCCESS}